

Department of Computer Science

Moderne Softwarearchitektur

MSA26-PeerReview-Gruppe-A

Abgabedatum:

17.07.2026

Eingereicht von:

Matthias Matthies

Matr.-Nr.: 106971
stud106971@fh-wedel.de

Erika Musterfrau

Matr.-Nr.: 654321
stud654321@fh-wedel.de

Marcel Ossig

Matr.-Nr.: 106970
stud106970@fh-wedel.de

John Doe

Matr.-Nr.: 987654
stud987654@fh-wedel.de

Betreut von:

Prof. Dr. Ulrich Hoffmann

Fachhochschule Wedel
Feldstraße 143
22880 Wedel
Tel: 04103 - 8048 - 41
urlich.hoffmann@fh-wedel.de

Table of Content

Abkürzungsverzeichnis	III
1 Einleitung	1
2 Features der App	2
3 Architektur	3
3.1 Übersicht	3
3.2 Infrastruktur Architektur	5
3.2.1 Compute	6
3.2.2 Netzwerk	7
3.2.3 Datenbank Architektur	9
3.2.4 User Management	10
3.3 Configuration Service (Plugin Service)	11
3.4 Server-Sent Events im Communication und Notification Service	11
3.5 Web-UI	11
3.6 DevOps und Infrastructure as Code	12
4 Umsetzung und Workflow	14
4.1 Baseline	14
4.2 Agentic Coding	15
4.2.1 Agents.MD / Claude.MD	15
4.2.2 Antigravity	16
4.2.3 Claude Code	16
4.2.4 OpenCode	16
4.2.5 GitHub Copilot Reviewer	16
4.3 CI / CD Integration	16
4.4 Testen (Manuell)	17
4.5 Optimierung	17
4.6 Dokumentation	17
4.6.1 Postman Collection	17
4.6.2 AI Written Documentation	17
5 Kritik und Einordnung des Vibecoding	18
5.1 Was lief gut	18
5.2 Was lief eher schlecht	18
5.3 Herausforderungen	18
5.4 Zukunfts-„Ausrichtung“	18
6 Kurze Zusammenfassung	19
6.1 Zusammenfassung	19
6.2 Fazit	19
6.3 Ausblick	19
A Appendix	20

A.1 Unterteilung der Annotations-Attribute	20
Bibliography	21
Declaration of Authorship	22

Abkürzungsverzeichnis

AWS – Amazon Web Services

BFF – Backend for Frontend

CDN – Content Delivery Network

LLM – Large Language Model

MVP – Minimum Viable Product

SPA – Single Page Application

1 Einleitung

Ich habe mal 1 Teil eines Kapitels aus meiner Seminararbeit in diesem template gelassen damit ihr gucken könnt, wie man etwas in Typst macht. Nachdem wir die Struktur abgeklärt haben, würde ich die dazugehörigen Dateien erstellen.

2 Features der App

3 Architektur

Einen Kernpunkt dieser Ausarbeitung bildet die zugrunde liegende Softwarearchitektur. Um eine ausfallsichere und skalierbare Software zu entwickeln, fanden durchgehend moderne Architekturmuster Anwendung. Diese Muster wurden auf allen Ebenen des Systems berücksichtigt, von der grundlegenden Klassifizierung als Microservices-Architektur bis hin zur Nutzung spezifischer Webprotokolle wie Server-Sent Events (SSE) für die asynchrone Kommunikation. Im Folgenden werden die verschiedenen Architekturebenen detailliert dargestellt sowie die zugrunde liegenden Motivationen und die daraus resultierenden technischen Einschränkungen beleuchtet.

3.1 Übersicht

Die entwickelte Software implementiert eine Microservices-Architektur, bei der domänenspezifische Fachlichkeiten nach dem Prinzip der losen Kopplung gekapselt und in eigenständige Services ausgelagert wurden. Grundlage hierfür war eine detaillierte Analyse des gesamten Peer-Review-Prozesses, aus der sich spezifische fachliche Schwerpunkte (Bounded Contexts) ableiten ließen.

Insgesamt wurden acht Domänen identifiziert. Abbildung 1 bietet einen Überblick über alle Services sowie deren Relationen zueinander. Vier dieser Services bilden das Kern-Framework des eigentlichen Review-Prozesses, während vier weitere Services zusätzliche allgemeine oder technisch bedingte Domänen abdecken.

Die Services des Frameworks umfassen den **Configuration Service**, **Matching Service**, **Submission Service** und **Response Service**. Die Architektur des Erstgenannten wird im nachfolgenden Abschnitt 3.3 genau erläutert. Grundsätzlich besteht dessen Fachlichkeit darin, verschiedene Einreichungstypen sowie Review-Formate bereitzustellen und das Erstellen neuer Abgaben zu verwalten. Nach dem erfolgreichen Anlegen einer Abgabe ermittelt der Matching Service eine passende Kombination aus Abgabe und Reviewer, wobei die allgemeine Verfügbarkeit sowie die jeweiligen Fachgebiete berücksichtigt werden. Anschließend ermöglicht der Submission Service die eigentliche Abgabe der Arbeit durch den Autor mittels PDF-Upload. Abschließend nimmt der Response Service die Gutachten der Reviewer entgegen und stellt diese nach Abschluss des gesamten Prozesses dem Autor zur Verfügung.

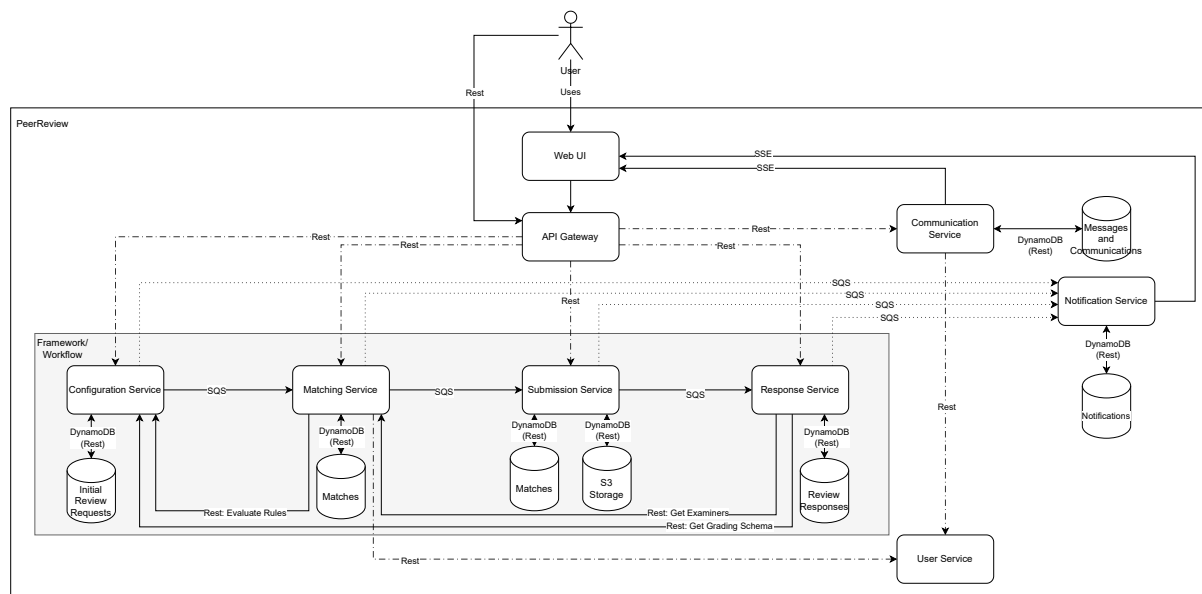


Abbildung 1: Applikationsarchitektur

Auffällig ist der Verzicht auf einen zentralen Orchestrations-Service (Orchestrator), welcher in Microservices-Architekturen häufig zum Einsatz kommt. Diese Entscheidung wird durch die Natur des Peer-Review-Verfahrens begünstigt: Es handelt sich um einen deterministischen Workflow, bei dem nach jedem abgeschlossenen Schritt eindeutig der nächste folgt. Die einzelnen Services des Frameworks können somit über eine ereignisgesteuerte Architektur (Event-Driven Architecture) mittels asynchroner Message-Queues direkt miteinander kommunizieren.

Die zusätzlichen Services umfassen den **Communication Service**, **Notification Service**, **User Service** sowie die **Web-UI**. Der Communication Service ermöglicht den interaktiven Austausch der Nutzer über die Plattform, entweder in Form von Direktnachrichten zwischen zwei Personen oder als fachlicher Austausch über eine spezifische Abgabe mit mehreren Beteiligten. Diese Funktion berücksichtigt strikt die Vorgaben des Configuration Service, der je nach Review-Format (z. B. Double-Blind-Verfahren) jegliche direkte Kommunikation unterbinden kann. Der Notification Service ist für die Benachrichtigung der Nutzer verantwortlich. Diese Benachrichtigungen werden vom System basierend auf spezifischen Ereignissen im Review-Prozess generiert und dem Anwender asynchron ausgespielt. Der User Service bietet eine einheitliche Schnittstelle für das Identity- und Access-Management, wobei dieser Service eine Abstraktionsschicht („Wrapper“) für AWS Cognito darstellt, was im Abschnitt 3.2.4 detailliert beschrieben wird. Die Web-UI fungiert als Client-Side-API-Aggregation: Sie konsumiert sämtliche REST-APIs der Backend-Services und bündelt diese für den Endnutzer.

Die technologische Basis bilden im Backend Java 25 mit dem Spring Boot Framework sowie React mit Vite für das Frontend. Die Infrastruktur wird vollständig über Amazon Web Services (AWS) abgebildet und im Abschnitt 3.2 näher beleuchtet. Der gewählte

Microservices-Ansatz erlaubt prinzipiell eine polyglotte Entwicklung, bei der beliebige Technologien eingesetzt werden können, sofern die API-Schnittstellen kompatibel bleiben. Zu diesem Zweck stellen alle Backend-Services ihre Funktionalitäten über vollständig dokumentierte REST-APIs auf Basis von OpenAPI-Spezifikationen bereit.

Die Backend-Services lassen sich als funktionale Basis-Services kategorisieren. Sie umfassen bewusst eng gefasste Fachlichkeiten und bilden isoliert betrachtet keinen eigenständigen, vollständigen Geschäftsprozess ab. Erst durch die Integration in der zustandslosen Web-UI, die als aggregierender Service agiert, verschmelzen die isolierten Backend-APIs zu einem zusammenhängenden Prozess. Dieser Ansatz der clientseitigen Bündelung bietet den Architekturvorteil, dass die Web-UI hochgradig austauschbar ist. Es können jederzeit neue Frontends entwickelt werden, die basierend auf den vorhandenen REST-Schnittstellen einen modifizierten Detaillierungsgrad oder alternative User Journeys abbilden, ohne die Backend-Logik anzupassen.

3.2 Infrastruktur Architektur

Die Architektur basiert auf containerisierten Deployments der einzelnen Microservices. Die gesamte Bereitstellung der Compute-Ressourcen sowie weiterer Plattform-Dienste erfolgt über AWS. Ein maßgebliches Entwurfskriterium war eine strikte Kosteneffizienz, weshalb ausschließlich moderne Serverless-Komponenten zum Einsatz kommen. Zugunsten der Kostenoptimierung wurde auf eine ausgedehnte High-Availability-Architektur (Multi-AZ) vorerst verzichtet, eine nachträgliche Implementierung ist jedoch möglich.

Abbildung 2 veranschaulicht die eingesetzten AWS-Dienste sowie deren strukturelles Zusammenwirken innerhalb der Software. Die genaue Funktionsweise dieser Komponenten wird in den nachfolgenden Abschnitten detailliert beschrieben.

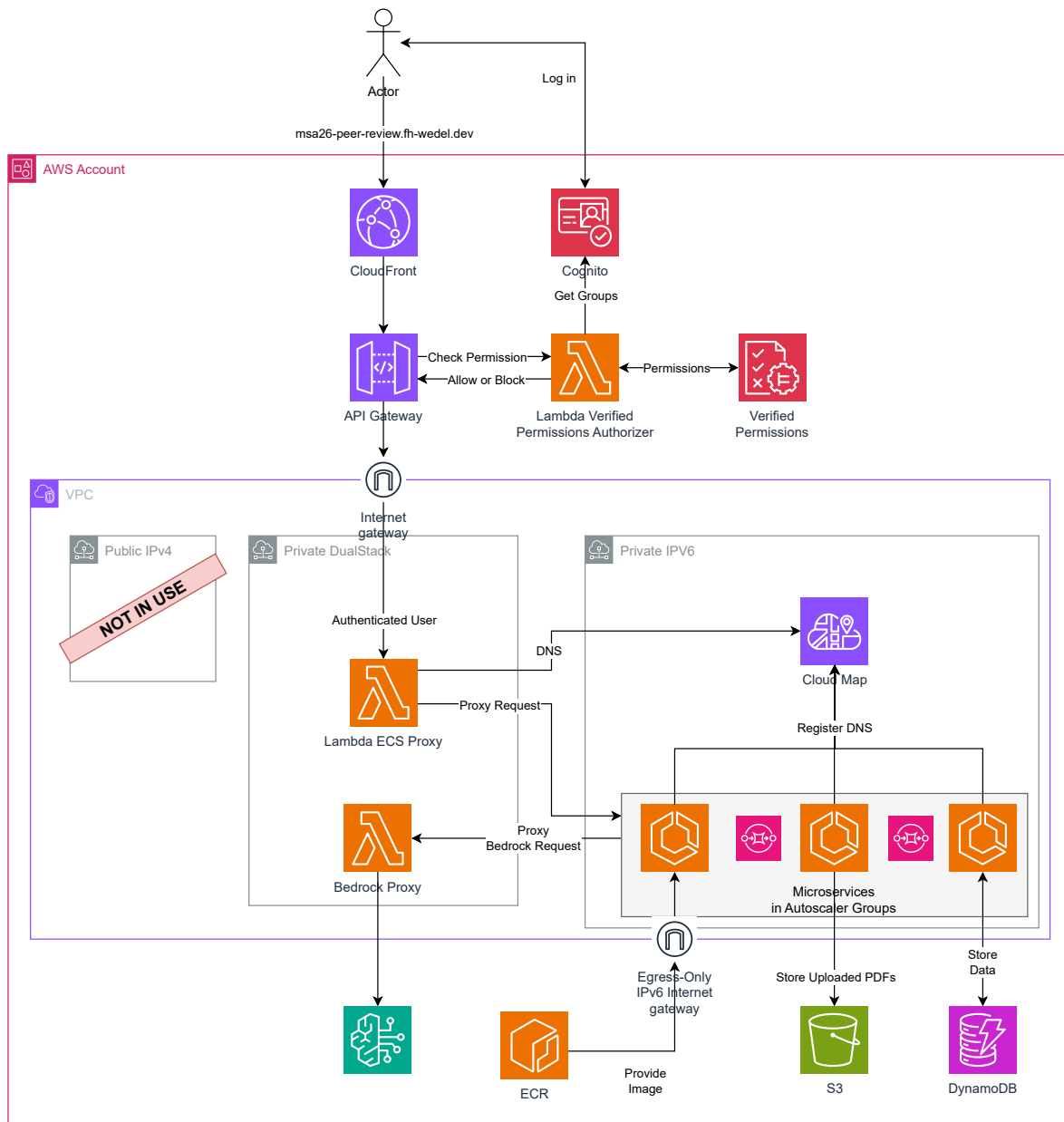


Abbildung 2: Infrastrukturarchitektur (AWS)

3.2.1 Compute

Die Bereitstellung der Rechenleistung (Compute) erfolgt über den AWS Elastic Container Service (ECS) in Kombination mit der Serverless-Compute-Engine AWS Fargate. Fargate ermöglicht die vollautomatisierte Provisionierung von Laufzeitumgebungen für Docker-Container, deren Images in der Amazon Elastic Container Registry (ECR) versioniert vorliegen.

Die Ausfallsicherheit der Services ist systembedingt hoch und kann durch vertikales Skalieren weiter optimiert werden. Zur Bewältigung von Lastspitzen kommt ein Autoscaler zum Einsatz, der mittels einer Target-Tracking-Skalierungsrichtlinie (Target Tracking

Policy) eine durchschnittliche CPU-Auslastung von 75 % anstrebt. Bei Überschreiten dieses Schwellenwerts wird vertikal hochskaliert, bei geringerer Last entsprechend herunterskaliert. Auch das kontinuierliche Ausrollen neuer Software-Versionen wird durch AWS ECS Fargate nativ unterstützt, was moderne Deployment-Strategien wie Blue/Green-, Canary- oder Rolling-Deployments ermöglicht. Hierbei werden neue Service-Instanzen parallel gestartet und erst nach erfolgreicher Gesundheitsprüfung (Health Check) für den produktiven Datenverkehr freigegeben.

Zur weiteren Kostenreduktion nutzen alle ECS-Services die ARM64-Prozessorarchitektur (AWS Graviton), welche ein besseres Preis-Leistungs-Verhältnis aufweist als vergleichbare x86-Architekturen. Zudem werden AWS Spot-Instanzen verwendet, die erhebliche Rabatte bieten, jedoch durch AWS bei Kapazitätsengpässen kurzfristig terminiert werden können. Für den aktuellen Projektstatus ist dies ein idealer Kompromiss, da kurze Downtimes tolerierbar sind und der Autoscaler ausfallende Instanzen automatisch durch neue ersetzt. Für einen ausfallsicheren Produktivbetrieb könnte das System auf eine Multi-Availability-Zone-Strategie (Multi-AZ) erweitert werden, um den Ausfall einzelner Rechenzentren transparent zu kompensieren. Aufgrund der damit verbundenen Mehrkosten wurde dies jedoch im aktuellen Scope nicht umgesetzt.

Da jede Instanz mit der kleinstmöglichen Konfiguration von 0,256 vCPUs und 512 MB RAM dimensioniert ist, belaufen sich die reinen Compute-Kosten, unter der Annahme, dass kein Autoscaling stattfindet und kontinuierlich nur eine Instanz läuft, auf 0,078192 € pro Tag. In Summe resultiert dies in Kosten von 0,625536 € pro Tag für alle Services, was die Vorgabe einer hocheffizienten Infrastruktur vollumfänglich erfüllt.

Die Compute-Instanzen kommunizieren größtenteils asynchron und entkoppelt über den Amazon Simple Queue Service (SQS). Hierbei wurde das Paradigma der Data Ownership strikt angewendet: Jeder Service stellt eine dedizierte SQS-Queue als Eingangskanal (Input Queue) bereit. Andere Services, die Daten an diesen Service übergeben möchten, schreiben ihre Events direkt in die jeweilige Ziel-Queue.

Bestimmte fachliche Abläufe erfordern jedoch eine synchrone Kommunikation, welche direkt über REST abgebildet wurde. Dafür nutzen die Services interne DNS-Namen zur Service Discovery über CloudMap, deren Funktionsweise im folgenden Abschnitt zum Netzwerk erläutert wird.

3.2.2 Netzwerk

Netzwerkseitig wird die Software in einer Virtual Private Cloud (VPC) überwiegend in einem sicheren, privaten Subnetz bereitgestellt. Die Netzwerkkonfiguration ist ein kritischer Aspekt der Kostenoptimierung, da insbesondere passiv bereitgestellte Netzwerkinfrastruktur in der AWS-Cloud erhebliche Fixkosten verursachen kann.

Durch das Deployment in ein privates Subnetz sind die Instanzen nicht über das öffentliche Internet routingfähig, was die Sicherheit der Architektur signifikant erhöht. Dies bedeutet jedoch auch, dass die ECS-Instanzen ohne weitere Maßnahmen nicht nach außen kommunizieren können, um beispielsweise initiale Docker-Images aus der ECR zu pullen. Ein klassischer Lösungsansatz bestünde in der Provisionierung eines von AWS verwalteten NAT Gateways (Network Address Translation). Ein NAT Gateway verursacht jedoch hohe passive Kosten von ca. 50 € pro Monat, zuzüglich variabler Traffic-Kosten. Eine Alternative wäre die Zuweisung öffentlicher IPv4-Adressen an jede ECS-Instanz. Da AWS jedoch Gebühren für die Nutzung öffentlicher IPv4-Adressen erhebt (ca. 3,50 € pro IP/Monat), würden sich die monatlichen Kosten bei acht Services auf knapp 30 € summieren.

Die implementierte Lösung nutzt stattdessen ein reines IPv6-Netzwerk. AWS ermöglicht es, einem IPv6-Subnetz ein kostenfreies Egress-Only-Internet-Gateway zuzuordnen. Dadurch können die ECS-Instanzen ausgehende Verbindungen über IPv6 aufbauen, um Abhängigkeiten oder ECR-Images herunterzuladen. Obwohl AWS auch Dual-Stack-Subnetze (parallele IPv4- und IPv6-Nutzung) anbietet, war dieser Ansatz nicht zielführend. In einem Dual-Stack-Setup nutzen ECS-Instanzen den Image-Pull über IPv4, was ohne NAT Gateway oder öffentliche IPv4-Adresse zwangsläufig fehlschlägt. Der konsequente Einsatz eines reinen IPv6-Subnetzes löst dieses Problem kostenneutral, schafft jedoch neue Herausforderungen beim Routing des eingehenden Datenverkehrs (Ingress).

Externe Requests müssen die ECS-Instanzen erreichen, optimalerweise nicht über IPv6-Adressen, sondern über DNS. Hierfür kommt das AWS API Gateway zum Einsatz, das eine öffentlich verfügbare DNS-Auflösung ohne Fixkosten bereitstellt. Die technische Limitierung besteht jedoch darin, dass das API Gateway ECS-Instanzen in einem reinen IPv6-Netzwerk nicht direkt als Ziel (Target) adressieren kann, da es zwingend IPv4-Endpunkte erwartet. Als Lösung wurden AWS Lambda-Funktionen als Proxy-Schicht entwickelt und in einem separaten, Dual-Stack-fähigen Subnetz bereitgestellt. Das API Gateway routet eingehende Anfragen an diese Lambda-Proxies, welche die Requests an die ECS-Services im IPv6-Subnetz weiterleiten. Die Lambdas können in einem Dual-Stack-Subnetz betrieben werden, da sie keine externen Docker-Images über IPv4 beziehen müssen und die IPv4-Netzwerkstrecke zum API Gateway systemseitig gewährleistet ist.

Für dieses Proxy-Pattern müssen die Lambda-Funktionen die internen, dynamischen IPv6-Adressen der ECS-Services kennen. Zur Realisierung der Service Discovery kommt AWS Cloud Map zum Einsatz. Alle ECS-Services registrieren sich beim Bootvorgang vollautomatisch bei Cloud Map. Die Lambda-Proxies fragen den internen DNS-Namen über Cloud Map ab, erhalten die aktuelle IPv6-Adresse des Ziels und leiten den Request entsprechend weiter.

Dieser Lambda-Proxy-Ansatz kann durch sogenannte Kaltstarts (Cold Starts) zeitweise zu erhöhten Latenzen führen, da die Initialisierung der Lambda-Laufzeitumgebung einige Sekunden beanspruchen kann. Eine performantere, aber kostenintensive Lösung ist die Nutzung von Provisioned Concurrency, bei der vorkonfigurierte Lambda-Instanzen warmgehalten werden. Diese Funktion wird im Projekt jedoch nur temporär zu Demonstrations- und Testzwecken aktiviert. Alternativ ließe sich ein Application Load Balancer (ALB) implementieren, der die Lambda-Proxies oder gar das API Gateway ersetzt, eine feste IPv4-Adresse bereitstellt und Lastverteilungs-Metriken nativ unterstützt. Die fixen Vorhaltekosten eines ALBs belaufen sich jedoch auf ca. 16 € pro Monat. Bei einer dedizierten Bereitstellung pro Service würden somit monatliche Fixkosten von rund 130 € entstehen, was den ökonomischen Vorgaben des Projekts widerspricht.

Ein weiterer Lambda-Proxy wurde zwischen dem Response Service und AWS Bedrock implementiert, da Bedrock derzeit keine nativen IPv6-Verbindungen unterstützt. Durch das Proxying über eine Dual-Stack-fähige Lambda-Funktion konnte diese Einschränkung erfolgreich umgangen werden.

Den Abschluss der Netzwerkarchitektur bildet Amazon CloudFront als Content Delivery Network (CDN). CloudFront reduziert die globale Latenz durch Edge-Caching und leitet Anfragen effizient an den geografisch nächsten Edge-Standort weiter. Über CloudFront wurde zudem die Custom Domain `fh-wedel.dev` konfiguriert. Anstatt die flüchtigen Endpunkte des API Gateways direkt anzusprechen, ermöglicht Amazon Route 53 eine feste DNS-Auflösung, sodass externe Anfragen konsistent über die Custom Domain an das API Gateway geroutet werden.

3.2.3 Datenbank Architektur

Die Datenbankarchitektur unterliegt denselben strengen Vorgaben zur Kosteneffizienz. Es war zwingend erforderlich, eine vollständig serverlose Datenbanklösung ohne Fixkosten zu implementieren. Da die Geschäftsdaten keine strenge relationale Modellierung erforderten, fiel die Wahl auf Amazon DynamoDB, eine hochskalierbare Key-Value-Datenbank mit einem Pay-per-Request-Preismodell.

Um eine starke strukturelle Kopplung zu vermeiden und die Unabhängigkeit der Microservices zu wahren, wurde das Database-per-Service-Pattern konsequent umgesetzt. Jeder Service, der Persistenz benötigt, verfügt über eine eigene, isolierte DynamoDB-Tabelle.

Bei der Datenmodellierung kam konsequent ein modernes Single-Table-Design zum Einsatz. Dieser Ansatz ermöglicht es, alle zu einer Abgabe gehörenden Entitäten eines Services über eine einzige Query hocheffizient abzufragen. Komplizierte Joins über mehrere Tabellen hinweg oder das Absetzen sequenzieller Abfragen (N+1-Problem) entfallen dadurch vollständig.

Zudem wird das Prinzip der Datenhoheit (Data Sovereignty) strikt eingehalten: Jeder Service speichert ausschließlich die Daten, die in seinen Bounded Context fallen. Dies verhindert Datenredundanz über mehrere Services hinweg und eliminiert das Risiko von Dateninkonsistenzen bei Updates. Benötigt ein Service aggregierte Daten, fragt er diese zur Laufzeit über die REST-Schnittstelle des zuständigen Data Owners an.

3.2.4 User Management

Für das Identitäts- und Zugriffsmanagement (IAM) werden AWS Cognito und Amazon Verified Permissions (AVP) genutzt. Cognito agiert als Managed Identity Provider (IdP) und übernimmt komplexe Aufgaben wie die Bereitstellung gehosteter Login-Seiten, das Management von Anmeldeinformationen (Credentials) sowie die Verwaltung von Session-Tokens. Dies entspricht Best Practices der modernen Softwareentwicklung, da sich das Entwicklungsteam auf die Kernkompetenzen und die Geschäftslogik konzentrieren kann. Zudem ermöglicht Cognito die Gruppierung von Nutzern, was die Grundlage für eine rollenbasierte Zugriffskontrolle (Role-Based Access Control, RBAC) bildet.

Zur Autorisierung der API-Gateway-Endpunkte kommt Amazon Verified Permissions zum Einsatz. AVP definiert feingranulare Policies, die den Zugriff auf Ressourcen regulieren, sodass nur berechtigte Cognito-Benutzergruppen bestimmte API-Endpunkte aufrufen dürfen. Die Durchsetzung dieser Richtlinien erfolgt über einen Lambda Authorizer, der vom API Gateway bei jedem eingehenden Request aufgerufen wird. Diese Lambda-Funktion validiert das Session-Token, gleicht die Cognito-Gruppen des Nutzers mit den in AVP hinterlegten Policies ab und trifft die finale Autorisierungsentscheidung (Allow/Deny).

Auch diese Architektur erzeugt keine passiven Kosten, und die nutzungsbasierten Gebühren für Cognito und AVP sind marginal. Der einzige Trade-off besteht in potenziellen Latenzen durch Kaltstarts des Lambda Authorizers. Auch hier kann durch Provisioned Concurrency die Antwortzeit signifikant optimiert werden, ein Effekt, der sich potenziert, wenn parallel auch der Lambda-Proxy vorkonfiguriert ist.

Zusätzliche Sicherheitsmechanismen sind direkt in der Geschäftslogik (Applikationsschicht) verankert. Hierbei entscheidet die Software kontextbezogen, ob ein Nutzer bestimmte Datensätze lesen oder mutieren darf (z. B. Author einer Abgabe). Diese Logik wird über Spring Security abgebildet, wobei die aus AWS Cognito injizierten Benutzernamen und Rollen als vertrauenswürdige Quelle (Single Source of Truth) dienen.

3.3 Configuration Service (Plugin Service)

3.4 Server-Sent Events im Communication und Notification Service

Eine architektonische Besonderheit des Communication und Notification Services ist die Implementierung von Server-Sent Events (SSE). SSE etabliert eine unidirektionale Verbindung vom Server zum Client, über die der Server asynchron Live-Updates und Benachrichtigungen pushen kann. Im Vergleich zu WebSockets, die bidirektional und in der Handhabung deutlich komplexer sind, stellt SSE für diesen Anwendungsfall eine leichtgewichtigere Alternative dar. Auch gegenüber clientseitigem Polling (z. B. Short oder Long Polling) bietet SSE den Vorteil, dass Updates in Echtzeit und mit stark reduziertem Overhead (ohne ständigen Verbindungsaufbau) übertragen werden.

Der Einsatz von SSE hat sich für die Chat- und Benachrichtigungsfunktionen als äußerst zuverlässig erwiesen. Es existiert jedoch eine signifikante infrastrukturelle Einschränkung durch das AWS API Gateway, welches ein hartes Timeout von 29 Sekunden für aktive Verbindungen erzwingt. Für eine langlebige SSE-Verbindung ist dies im Standardbetrieb nicht praktikabel.

Um dieses Limit zu umgehen, beendet der Server die SSE-Verbindung proaktiv nach 25 Sekunden. Da das SSE-Protokoll nativ über Selbstheilungsmechanismen (Auto-Reconnect) verfügt, erkennt der Client den Verbindungsabbruch sofort und baut die Verbindung automatisch neu auf. Dadurch wird ein scheinbar unterbrechungsfreier Live-Datenstrom simuliert.

3.5 Web-UI

Der Web-UI-Service ist für die Bereitstellung der Weboberfläche des PeerReview-Systems verantwortlich. Sie nimmt in der komponenten- und ereignisgesteuerten Gesamtanwendung eine zentrale Rolle ein, da sie zur Ermöglichung von Benutzerinteraktionen, Daten bereitstellen muss, welche auf diverse Services verteilt vorliegen. Hierdurch wird einerseits die Skalierung von einzelnen Services ermöglicht, andererseits wird jedoch gleichzeitig eine Datenaggregation erforderlich, bevor die Weboberfläche vollständig angezeigt werden kann. Angesichts des Umfangs des Projekts, wurde sich in diesem Zusammenhang dazu entschieden, kein dediziertes Backend for Frontend (BFF) zu implementieren, welches diese Aufgabe übernehmen könnte. Folglich obliegt es dem Web-UI, die notwendigen Daten von den verschiedenen Services abzurufen und zusammenzuführen. Dies soll dazu beitragen, die Weboberfläche weiter zu entkoppeln und infrastrukturelle Komplexität gemäß der Idee zur Implementierung eines Minimum Viable Product (MVP) zu reduzieren. Die Weboberfläche ist zustandslos und verwendet hiervon abweichend lediglich den LocalStorage des Browsers, um Einstellungen wie den präferierten Ansichtsmodus des Benutzers zu speichern.

Konzeptionell erklärt sich aus diesen Bedingungen, weshalb die Weboberfläche als Single Page Application (SPA) unter Verwendung von React und Vite implementiert wurde. Dies ermöglicht es, Daten von verschiedenen Services asynchron abzurufen und die Benutzeroberfläche dynamisch und schrittweise zu aktualisieren, ohne dass eine vollständige Seitenaktualisierung erforderlich wird. Die zusätzliche Verwendung eines CDN erlaubt es zudem die Latenzzeit der Weboberfläche zu reduzieren.

Zur Kommunikation mit den Backend-Services werden von der Web-UI ausschließlich solche API-Clients verwendet, welche anhand der OpenAPI-Spezifikationen der jeweiligen Services automatisiert generiert wurden. Der Aufbau des Projekts als Mono-Repository trägt hierbei dazu bei, dass die Schnittstellen leicht überprüfbar bleiben und inkompatible Änderungen frühzeitig bemerkt werden. Anpassungen an einem Service erfordern lediglich die Neugenerierung des jeweiligen Clients. In diesem Zuge wurde auf die explizite Einführung von Data-Access-Objekten innerhalb der Weboberfläche verzichtet. Sollte die Komplexität des Projekts durch zukünftige Erweiterungen steigen oder externe Services angebunden werden, sollte dies erneut evaluiert werden.

Die Weboberfläche ist aufgrund der Verwendung von AWS Cognito nicht für die Bereitstellung einer Anmeldeseite zuständig. Stattdessen wird die Authentifizierung durch einer Umleitung auf eine von AWS angebotenen Anmeldeseite durchgeführt. Nach erfolgreicher Authentifizierung wird der Benutzer zurück auf die Weboberfläche geleitet, wobei ein Session-Token übergeben wird, welches für die Dauer der Sitzung gültig ist. Dieses Token wird von der Weboberfläche bei jedem Request an die Backend-Services übermittelt, um die Identität des Benutzers zu verifizieren und den Zugriff auf die jeweiligen Ressourcen zu autorisieren. Gleiches gilt für den Logout, bei welchem der Benutzer kurzzeitig ebenfalls umgeleitet wird, um die Sitzung zu beenden.

3.6 DevOps und Infrastructure as Code

Zu einer modernen Softwarearchitektur gehören neben dem reinen Applikations- und Infrastrukturdesign zwingend auch DevOps-Praktiken, die ein kontinuierliches Testen, Bauen und Ausrollen (CI/CD) der Software ermöglichen. Die Architektur der CI/CD-Pipeline ist modular aufgebaut, wodurch sich neue Services aufwandsarm integrieren lassen. Der genaue Entwicklungszyklus wird in Abschnitt 4.3 detailliert beschrieben.

Um das Deployment in die AWS-Cloud vollautomatisiert durchzuführen, wurde Infrastructure as Code (IaC) angewandt. Die Infrastruktur wird dabei in deklarativem Code beschrieben, welcher innerhalb der Pipeline ausgeführt wird, um die entsprechenden Cloud-Ressourcen zu provisionieren oder zu aktualisieren. Zum Einsatz kam hierbei das AWS Cloud Development Kit (CDK) in der Programmiersprache TypeScript.

Auf Code-Ebene nutzen die IaC-Komponenten eine eigens entwickelte **InfraLibrary**. Diese abstrahiert und standardisiert komplexe Infrastruktur-Setups, wie das Deployment eines ECS-Services inklusive Lambda-Proxy, API-Gateway-Routen und AVP-Integration, in wenigen, wiederverwendbaren Zeilen Code. Durch diese Abstraktion wird sichergestellt, dass alle Services auf identischen infrastrukturellen Mustern basieren. Dies gewährleistet eine hohe Konsistenz, erleichtert das Debugging enorm und ermöglicht es, infrastrukturelle Anpassungen zentral vorzunehmen und global auf alle Services auszurollen.

Ergänzend zur InfraLibrary existiert das Projekt **InfraBaseline**. Dieses provisioniert die grundlegende Netzwerkinfrastruktur, den AWS-Cognito-Userpool, die Cloud-Map-Namespaces sowie die CloudFront-Distribution.

Der massive strategische Vorteil von IaC zeigte sich im Projektverlauf mehrfach. So konnten Features isoliert getestet und die Infrastruktur jederzeit deterministisch auf ältere Stände zurückgerollt werden, wobei Applikationscode und Infrastruktur stets konsistent blieben. Besonders erfolgskritisch war der IaC-Ansatz, als der ursprüngliche AWS-Account aufgrund eines abgelaufenen Leases gesperrt wurde. Dank der vollständigen Deklaration im Code konnten sämtliche Ressourcen nahezu ohne manuelle Eingriffe in einem neuen Account hochgefahren werden. Ohne IaC hätte dieser Vorfall eine stundenlange, extrem fehleranfällige manuelle Neuprovisionierung erfordert.

4 Umsetzung und Workflow

Ein wesentlicher Bestandteil der praktischen Umsetzung umfasste den Einsatz von KI-gestützten Coding-Agenten wie Claude Code oder Google Antigravity. Vor Beginn der eigentlichen Projektphase wurden strategische Entscheidungen bezüglich deren Evaluierung getroffen, um eine bestmögliche Integration in den Entwicklungsprozess zu gewährleisten. Neben diesen modernen, Large Language Model (LLM)-basierten Ansätzen fanden klassische Verfahren des Software-Lebenszyklus Anwendung. Hierzu zählen die Einrichtung einer Continuous-Integration- und Continuous-Deployment-Pipeline (CI/CD) sowie das automatisierte und manuelle Testen.

4.1 Baseline

Zu Beginn der Implementierungsphase wurde die Entscheidung für ein Mono-Repository (Monorepo) getroffen. Die primäre Absicht bestand darin, den KI-Agenten den vollständigen Kontext über alle Microservices hinweg agnostisch bereitzustellen, ohne dass ein isolierter Wechsel zwischen verschiedenen Projektarchiven erforderlich ist. Obwohl es sich bei dem Vorhaben um eine vollständige Greenfield-Entwicklung handelte, wurde die initiale Baseline bewusst nicht agentisch, sondern lediglich KI-unterstützt und manuell entwickelt.

Im Rahmen dieser Baseline wurden alle grundlegenden Infrastrukturkomponenten mittels des AWS CDK implementiert. Dadurch konnte sichergestellt werden, dass die Services keine kostenintensiven Default-Ressourcen verwenden. Gleichzeitig förderte dieses Vorgehen das Systemverständnis innerhalb des Entwicklungsteams, was das spätere Debugging erheblich erleichterte. Da die Inter-Service-Kommunikation vollständig transparent war, ließ sich die Ursachenanalyse (Root-Cause-Analysis) im Fehlerfall schnell auf einzelne, isolierte Microservices eingrenzen. Dies erwies sich in einem verteilten System als fundamentaler Vorteil: Da autonome Agenten in komplexen AWS-Cloud-Infrastrukturen ohne vordefinierten Rahmen den Gesamtüberblick verlieren können, war es hocheffizient, den betroffenen Service vorab manuell zu identifizieren und den Agenten anschließend mit dedizierten Fehlermeldungen in einer isolierten Umgebung arbeiten zu lassen.

Ergänzend zur infrastrukturellen Basis wurde eine applikative Grundlage in Form eines **Template Service** geschaffen. Dieser diente den Coding-Agenten als struktureller Anker und Referenzarchitektur, um nahezu direkt lauffähige, neue Services zu generieren. Zu diesem Zweck wurde dieser Prototyp manuell implementiert, mit minimalen funktionalen Features ausgestattet und so auf die Infrastruktur abgestimmt, dass das Zusammenspiel aus Netzwerk, Autoscaling, DynamoDB-Datenbanken und dem API Gateway fehlerfrei funktionierte. Erst nach erfolgreicher Validierung dieser Basis wurden die Agenten

eingesetzt, um die eigentliche Geschäftslogik in Kopien dieses Template-Service zu implementieren.

Die Praxis zeigte, dass die Coding-Agenten die vorgegebene Baseline sowohl auf infrastruktureller als auch auf applikativer Ebene weitgehend adaptierten, was zu einer konsistenten Systemlandschaft führte. Dennoch war auffällig, dass die Agenten vereinzelt unvorhergesehene und nicht an die Architekturrichtlinien angepasste Entscheidungen trafen. Beispielsweise versuchte ein Agent eigenständig, eine PostgreSQL-Datenbank zu provisionieren, was für den vorliegenden Anwendungsfall fachlich nicht erforderlich war und zudem durch den Verzicht auf vollständige Serverless-Eigenschaften die Betriebskosten unnötig erhöht hätte.

Auch der Einsatz des Monorepos wirkte sich im Hinblick auf das Kontextfenster (Context Window) der verwendeten Sprachmodelle positiv aus, da die Modelle jederzeit serviceübergreifendes Systemwissen in ihre Generierung einbeziehen konnten. Aus ökonomischer Sicht offenbarte dieser Ansatz jedoch auch Nachteile: Bei der Initiierung neuer Prompts fiel das Kontextfeld und insbesondere die Anzahl der Input-Tokens sehr groß aus, was zu erhöhten API-Gebühren führte. Zudem besteht bei sehr großen Kontexten das inhärente Risiko von Qualitätseinbußen, da LLM dazu neigen, feine Details inmitten umfangreicher Datenmengen zu übersehen (Lost-in-the-Middle-Phänomen).

4.2 Agentic Coding

Unter Agentic Coding versteht man den Einsatz autonomer Software-Agenten auf Basis von LLMs, die über klassische Code-Vervollständigung hinausgehen. Im Gegensatz zu Assistenzsystemen versucht man mit Coding-Agenten einen kontinuierlichen Zyklus (Agentic Loop) aus Analyse, Planung, Umsetzung sowie anschließender Verifikation abzubilden. Dies soll die selbstständige Bearbeitung komplexer Entwicklungsaufgaben ermöglichen. Im Rahmen dieser Arbeit wurden verschiedene agentische Werkzeuge evaluiert, um deren Eignung für den Aufbau einer ereignisgesteuerten Microservice-Architektur zu untersuchen.

4.2.1 Agents.MD / Claude.MD

Um Coding-Agenten projektbezogene Instruktionen und Kontext bereitzustellen, etabliert sich zunehmend die plattformübergreifende `AGENTS.md` als offener Standard [1], der von Editoren wie Cursor und Frameworks wie Codex unterstützt wird. Im Gegensatz zu dieser herstellerunabhängigen Praxis weigert sich Anthropic mit Claude Code, diesen Standard zu integrieren, und verlangt stattdessen weiterhin die proprietäre Datei `CLAUDE.md`. Diese fehlende Interoperabilität erschwert den flexiblen Einsatz verschiedener Agenten im selben Projekt und stößt in der Entwickler-Community auf Kritik [2].

4.2.2 Antigravity

4.2.3 Claude Code

4.2.4 OpenCode

KI-Coding-Harnesses wie Claude Code, Antigravity oder Codex haben gemeinsam, dass sie üblicherweise für die Softwareentwicklung in Kombination mit den hauseigenen LLMs und Abonnements der jeweiligen Hersteller wie Anthropic, Google oder OpenAI vorgesehen werden. Eine Anbindung der Harnesses an LLMs von Drittherstellern oder selbstbetriebenen Instanzen wird nicht unterstützt oder obliegt vorbehaltlich zukünftigen Einschränkungen. Es wird aktiv versucht, die Verwendung der Abonnements außerhalb der jeweiligen Plattformen zu unterbinden und die Ökosysteme durch die Bindung an die proprietären Dienste zu stärken. Insbesondere Anthropic hat sich zum Beispiel bestrebt gezeigt, eine Verwendung mit nicht unternehmenseigener Software zu verhindern und den Zugriff von zuwiderhandelnden Benutzern vollständig zu sperren [3]. Eine Migration von einem Anbieter zu einem anderen wird hierdurch erheblich erschwert, obwohl die zur Anbindung von LLMs verwendeten Schnittstellen üblicherweise einheitlich sind.

Das Open-Source-Projekt OpenCode soll eine Alternative zu den proprietären Coding-Harnesses darstellen. Es bietet die Möglichkeit, beliebige LLMs von verschiedensten Anbietern zu integrieren oder die Agenten auf selbstgehosteten Instanzen zu betreiben. OpenCode ist hochgradig konfigurierbar und erlaubt die Erweiterung durch Plugins. So ist es beispielsweise möglich, dass KI Agenten, die auf verschiedenen Plattformen ausgeführt werden, miteinander kommunizieren und von einem einzigen kombinierten Harness orchestriert werden.

4.2.5 GitHub Copilot Reviewer

4.3 CI / CD Integration

Als etabliertes Verfahren der modernen Softwareentwicklung wurde eine Continuous-Integration- und Continuous-Deployment-Pipeline eingerichtet. Diese erlaubt ein automatisiertes Deployment der aktuellen Softwareversion auf Knopfdruck. Hierdurch werden fehleranfällige manuelle Bereitstellungsprozesse eliminiert. Gleichzeitig stellt der Ansatz der kontinuierlichen Integration sicher, dass Modifikationen frühzeitig zusammengeführt werden, wodurch potenzielle Integrationskonflikte oder Laufzeitfehler in einem frühen Stadium sichtbar werden. Neben der Option, spezifische Feature-Branches manuell zu deployen, wird bei jedem Merge auf den geschützten Hauptzweig (`main branch`) automatisch das Deployment aller vom Commit betroffenen Microservices initiiert.

Die Pipeline ist technologisch auf Basis von GitHub Actions realisiert. Der Workflow umfasst das automatisierte Kompilieren des Java-Backend-Codes sowie die Ausführung der Unit-Tests für die Applikation und den TypeScript-basierten IaC-Code. Nach erfolgreicher Verifikation wird das Docker-Image erstellt und in die Amazon Elastic Container Registry (ECR) gepusht. Im anschließenden Schritt initiiert die Pipeline mittels des Befehls `cdk deploy` das infrastrukturelle Deployment, wodurch sowohl die modifizierten Cloud-Ressourcen aktualisiert als auch die neuen Applikations-Images auf den ECS-Fargate-Instanzen ausgerollt werden.

4.4 Testen (Manuell)

4.5 Optimierung

4.6 Dokumentation

4.6.1 Postman Collection

4.6.2 AI Written Documentation

5 Kritik und Einordnung des Vibecoding

5.1 Was lief gut

Marcel Notitzen:

- Mono Repo dadurch gutes Kontext Wissen zumindest Möglich (Ob es auch immer geklappt hat sei dahin gestellt und war auch teurer dadruch)
- Sehr schnelles Entwickeln – Aber siehe nachteil – Sehr sehr sehr frustriernd
- ich glaube das Forntend ging gut
- An sich für normale Fragen sehr sher hlfreic und auch wenn man das xte mal das gleich macht. Aber das machen dann ja keine Agenten. Im Hinblick auf agenten eher negativ: Man hat kein plan was da raus kommt, wie das geht und man hat UNFASSBAR hohe Abhängigkeit/Vendor Lock in - Wenn die Preise stiegen dann hat man verloren

5.2 Was lief eher schlecht

Marcel Notitzen:

- Sehr frustierend: Keiner weiß wie die Applikation geht, man promopted ein fehler in die ki, wartet und hofft einfach nur das es passt. man hat überhaupt keine ahnugn ob ein fix funktioniert oder auch nicht
- Der Agent macht zum teil sehr sehr viel nur halb. Mal verhisst er die Test anzupassne, mal lässt sich das projekt gar nicht compilieren, wei das Imports fehlen etc. Selbst beim Propmt: Fixe comiler fehler und tests, hat er zwar änderungen geacmht aber es hat nicht funktioniert neue/andere fehler
- GitHub Reviwer: einmal hat er vorschläge gemacht, die wir angewadnt haben und dann hat alles nict h mehr funktioniereirt -> Schlechtes review wenn der funktionaliät danach nicht mehr geht

Luca:

- Die Landschaft an KI-Tools zur Entwicklung ist hochgradig verwirrend und trägt zur Frustration bei. Es gibt unzählige KI-Agenten-Harnesses, welche den gleichen Zweck erfüllen: Claude Code, Codex, Antigravity, GitHub Copilot, Cursor, OpenCode ... alle jeweils als CLI- und Dekstop-Variante. Jedes Tool ist etwas anders zu bedienen und zu konfigurieren. Alle paar Wochen gibt es etwas neues. Oftmals gibt es dazu noch Abonnements, die zur aktiven Verwendung der Tools abzuschließen sind.

5.3 Herausforderungen

5.4 Zukunfts-„Ausrichtung“

6 Kurze Zusammenfassung

6.1 Zusammenfassung

6.2 Fazit

6.3 Ausblick

A Appendix

A.1 Unterteilung der Annotations-Attribute

Bibliography

- [1] „AGENTS.md: A standard for AI coding agents instructions“. Zugegriffen: 2. Juli 2026. [Online]. Verfügbar unter: <https://agents.md/>
- [2] DylanLliii, „Feature Request: Support AGENTS.md.“. Zugegriffen: 2. Juli 2026. [Online]. Verfügbar unter: <https://github.com/anthropics/claude-code/issues/6235>
- [3] N. J. Engelking, „Anthropic is removing OpenClaw from its Claude subscriptions“. Zugegriffen: 1. Juli 2026. [Online]. Verfügbar unter: <https://www.heise.de/en/news/Anthropic-is-removing-OpenClaw-from-its-Claude-subscriptions-11246096.html>

Declaration of Authorship

I hereby declare ... ja was declaren wir denn?

Matthias Matthies

Erika Musterfrau

Marcel Ossig

John Doe